Thesis Progress Update

Eren Olmez

20 October 2026

1. From scalar routing to correctness routing

We used classical predictors with handcrafted features and lightweight convolutional neural networks (CNNs) with learned image features. Initially these predicted one scalar score; we later used 213 correctness outputs to exploit PERTINENCE's motivation: **models have complementary strengths**. Lower overall accuracy need not mean lower accuracy on every input. The target changed from one cost rank to candidate-specific success.

Data. Each router used a 5,000-image development pool with separate fitting/calibration subsets; 10,000 test images evaluated frozen policies. Classifiers used 45,000 training images. Deployed selection sees no test labels.

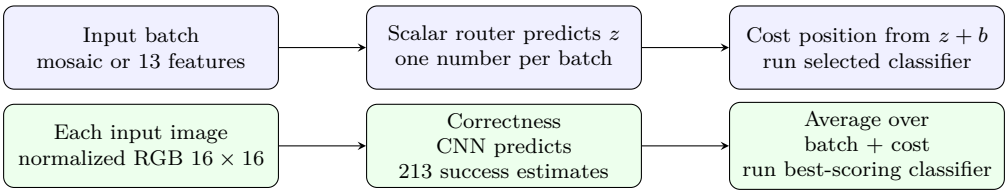
How we calculate the scalar training score.

1. Measure candidate costs at the chosen batch size. For latency, order models from fastest to slowest; for energy, order them from least to most GPU energy. This ordering uses *cost*, not accuracy.
2. For each labelled fitting image, run all candidates offline and compare their predicted classes with the true label. Mark each candidate correct or wrong.
3. Find the *cheapest candidate that was correct on that image*. If its zero-based position is r, the training target is $s = r/(M-1)$, where $M = 213$. If every candidate fails, use the reference model's position as fallback.
4. The direct batch target is the average of these per-image scores. The scalar CNN learns to predict that average using mean squared error (MSE); it does not receive the true labels at inference.

Hypothetical candidates, cheapest first	A	B	C	D	E
Zero-based cost position	0	1	2	3	4
Correct on this fitting image?	No	No	Yes	Yes	No

C is the cheapest correct model, so this image's target is $2/(5-1) = 0.50$. This is **not 50% confidence or accuracy**. Accuracy enters through the correct/wrong labels, and later through the overall accuracy constraint.

How the predicted score selects a model. With the trained router fixed, calibration tries 1,601 adjustments $b \in [-1, 1]$. A predicted score z selects position $\text{clip}(\lfloor (z+b)M \rfloor, 0, M-1)$ in the cost list. For each b, count the selected models' correct predictions on calibration images and estimate their cost. Keep the cheapest mapping satisfying reference accuracy minus the allowance; otherwise select the reference alone. With five candidates, $z = 0.46$ selects C when $b = 0$ and B when $b = -0.10$. The adjustment is frozen before testing.



Why change the target? Wrong/right/right and wrong/right/wrong share the same cheapest-correct rank but different alternatives. Average ranks need not identify a good batch classifier. For image x and candidate m, binary target $y_m(x)$ is 1 for correct, 0 for wrong. The CNN predicts success probability $p_m(x)$; q_m is its mean over the batch. Let C_m be candidate m's measured latency or energy at this batch size, $C_{\max}$ the largest candidate cost, and λ a calibrated cost penalty. Selection maximizes $q_m - \lambda C_m / C_{\max}$ over candidates. Sigmoid outputs estimate success, not CIFAR-100 classes, and need not sum to one.

Implemented architectures and fitting. Classical scalar inputs are 13 statistics: RGB mean/std (6), grayscale mean/std (2), gradient mean/std (2), edge density, saturation proxy and between-image brightness variation. Scalar TinyCNN uses 16 sampled 8×8 image tiles in a 32×32 mosaic: convolution $3 \to 16$, depthwise/pointwise $16 \to 24 \to 32 \to 48$, batch normalization (BN) and rectified linear activation (ReLU), global average pooling and a linear layer from 48 features to **one scalar cost-rank score per batch**. A separate calibrated mapping converts that score into a model choice. Training uses MSE, AdamW 0.001, weight decay 0.005 and 20 epochs. The correctness CNN uses each 16×16 image: $3 \to 16 \to 24 \to 32$, strided depthwise/pointwise blocks, ReLU, global pooling, 213 outputs without BN; mean binary cross-entropy, Adam 0.001, minibatches 64, 20 epochs. Three correctness fits serve seeds 42/45/48 with separate latency/energy calibration. Global scalar measurements stopped at 52 settings (stump/tree/logistic); its CNN was fitted/exported, not globally measured. The previous report completed 120 correctness-CNN settings.

2. Why the 1.5-point results change sharply

The smaller model at 1.5 points is the static comparator, not necessarily the router's choice. There are two separate decisions: the router chooses models for input batches; independently, validation chooses the cheapest eligible *fixed* model as the benchmark. Savings compare these two measured costs. Accuracy allowances are percentage-point losses (pp). A stricter allowance can force the benchmark to be expensive; relaxing it can make a much faster benchmark eligible even when routing does not change.

Exact example: latency, seed 48, batch 128. R101/R50/R34 denote ResNet-101/50/34; FP16/FP32 mean 16/32-bit floating point; b64 means batch size 64. At both 1 and 1.5 points, routing uses R101 FP16 for 7,424 images and R34 FP16 for 2,576. It does *not* switch to R50 FP16 at 1.5. R50 is the new *benchmark*: its calibration accuracy loss is 1.35 pp, so it first qualifies at 1.5 points.

Allowance	Acc.%	Route ms/mJ per image	Original benchmark	Original L% / E%	Fixed L% / E%
1.0	84.94	1.279 / 39.78	R101 FP16	+16.00 / +15.77	+16.00 / +15.77
1.5	84.94	1.277 / 39.69	R50 FP16	-47.66 / -48.21	+16.12 / +15.97
2.0	82.25*	0.653 / 20.35	R50 FP16	+24.49 / +24.61	+57.12 / +56.92

Fixed-benchmark columns: hold the 1-point R101 FP16 benchmark at 1.52280 ms/image and 47.2306 mJ/image for all three rows. These are recalculated savings against one frozen measured reference, not new matched measurements. The 1/1.5-point savings then stay near 16% for both costs: the large sign change in the original columns comes from the moving benchmark. At 1.5, $100(0.865 - 1.277)/0.865 = -47.66\%$: routing takes about 1.48 times the benchmark's time. The negative value is real underperformance against a better static alternative, not an arithmetic error. At 2 points the router favours R34 more and becomes faster, but accuracy falls below the required $85.80 - 2 = 83.80\%$ $(*)$. The positive saving therefore fails the accuracy requirement.

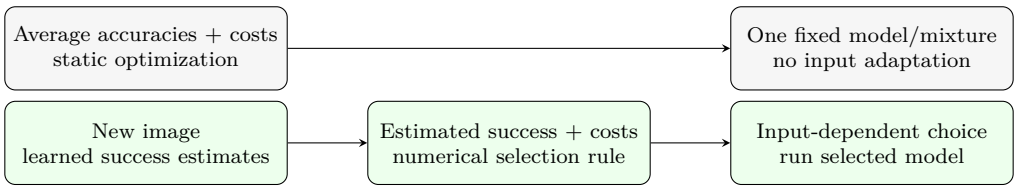
Why this repeats. Across the original 120 rows, eight settings flip from positive savings at 1 point to negative at 1.5: both objectives, seed 48, batches 16/32/64/128. All eight replace R101 FP16 with R50 FP16 as benchmark. Three turn positive again at 2 points (latency b64/b128, energy b128); *all three fail accuracy.* This is not a universal middle-budget anomaly: at b8 several budgets are negative throughout.

Other large changes have a measured runtime penalty. For latency/seed42/b8, the benchmark stays R101 FP16 from 1 to 1.5 points, but engine loads rise 217→313 and evictions 215→311; median routed time rises 69.46→99.81 s. Calibration uses static inference costs, which omit this load/eviction overhead. A looser allowance can thus select a worse end-to-end switching pattern. Smaller residual differences between identical policies remain unexplained measurement instability; they are not evidence of an improved policy.

Why machine learning rather than scalar optimization?

Answer: optimization chooses the action; machine learning (ML) supplies input-specific estimates. If only average model accuracy and cost are known, a simple constrained optimizer can select one fixed model, or fixed proportions of models, without looking at images. That is a valid baseline and does not need a neural router.

The extra question is: *for this new input, which model is likely to be correct?* Global accuracy cannot answer it: a model can be best overall yet weak on forest images. A learned router tries to recognize useful input patterns before paying for a large classifier. Its success estimates then enter an ordinary numerical optimization. ML and optimization are therefore complementary, not mutually exclusive; our scalar-score router was itself an ML predictor.



Justification must be empirical. ML is useful only if better selections compensate for feature extraction, router execution and cache/loading overhead at the same required accuracy. It must beat a validation-selected static model and simple non-neural rules, not merely uniform random among weak candidates. Current results do not establish that general advantage. META-DES trained a meta-classifier to estimate candidate competence and reported improved classification accuracy against existing dynamic-selection techniques on small-sample tasks (30 classification problems in the paper). Thus learned competence estimation was useful in that study. It did not establish latency or GPU-energy savings for our CIFAR-100/TensorRT setup; those must be demonstrated by our own measurements.[1]

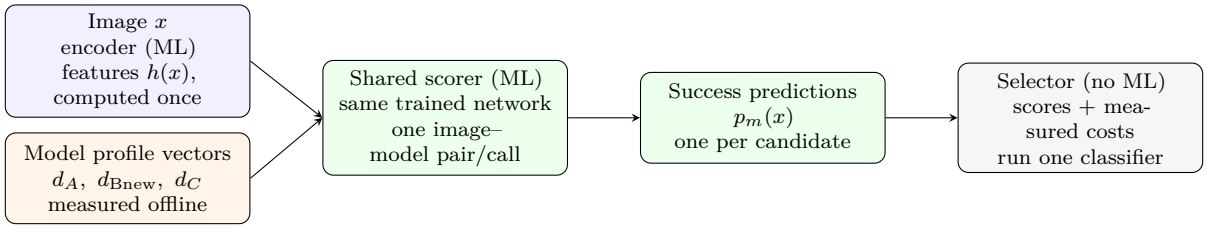
[1] Cruz et al., META-DES, *Pattern Recognition*, 2015; author manuscript.

3. Can a model be replaced without retraining the whole router?

Short answer. Deletion can avoid weight retraining; unseen additions are harder. The proposed shared scorer needs initial offline training and evidence about each new model. We have not demonstrated this extension.

Why the current router is tied to its candidates. Its 213 outputs have fixed meanings, such as success of R101 FP16 or R34 FP16. Replacing B by B-new leaves its output predicting *old B's* behaviour. Renaming the output does not teach it about B-new.

The proposed router has three parts. Write x for an input image and m for a candidate model. (1) A small *image encoder*, using machine learning (ML), returns learned image features $h(x)$. (2) A *shared scorer*, also ML, receives $h(x)$ and the candidate's stored profile vector d_m. It outputs $p_m(x)$, the estimated chance that model m predicts x correctly. (3) A *deterministic selector*, not ML, balances these estimates against measured costs. Together they form the router; the selected CIFAR-100 classifier is downstream. A, B-new and C below are example candidate identities.



What is a model profile? The vector d_m is a numerical description with the same fields for every candidate: for example, its validation accuracy on each of the 100 classes, its precision/pruning setting and measured latency/energy. It contains no label for the current unseen image. The same format lets the scorer process a new candidate without adding a model-specific output. A profile is only a summary; class-wise accuracy alone does not reveal exactly which individual images a model gets wrong.

How the scorer learns to produce a score. Offline, take labelled development image x_i (image number i) and run candidate A. If A predicts the correct class, assign $y_{iA} = 1$; otherwise assign 0. Repeat for other images and candidates. The notation $[h(x_i), d_A]$ means joining the encoder's features for image i with candidate A's profile; this combined vector is the scorer input. Its training target is y_{iA}. Fit the encoder/scorer with binary cross-entropy, using separate evidence to build profiles. Thus it learns associations between visual patterns, candidate characteristics and success. It does not calculate a reliable probability from a profile by itself, nor does it know unseen correctness.

Inference example (hypothetical). Suppose predicted success is A:0.70, B-new:0.85, C:0.80, with costs normalized by the largest available cost: 1.0,0.6,0.3. With penalty $\lambda = 0.10$, success minus penalized cost is 0.60,0.79,0.77. B-new wins and runs on the original input. For batch routing, average each candidate's image-success estimates before selection. Feature extraction and repeated scoring must count towards runtime.

If 50 models are removed. The current router can mask those 50 outputs and select among 163; the proposed router instead removes 50 profile entries and scorer calls. Neither necessarily needs weight retraining for deletion. Both need policy validation/recalibration, and neither can recover accuracy if essential capabilities were removed. Deletion alone is therefore not the main advantage of the shared design.

If a new model is added. Evaluate it on labelled development data, measure its costs, build its profile and insert it into the registry. The existing shared scorer can accept that profile without a new head; then validate its success predictions and recalibrate selection. This is *profiling/calibration*, not necessarily weight retraining, but it is still offline work. The proposed benefit is reusing the learned image–model scoring rule, rather than fitting a new named output for every replacement.

Where this proposal comes from. This is an adaptation of established pair-scoring and competence-estimation ideas, not a demonstrated new algorithm. Tornede et al.'s *Extreme Algorithm Selection with Dyadic Feature Representation* (2020) describes both problem instances and candidate algorithms using features. Our analogous pair is an image and a classifier profile. META-DES learns classifier competence for a query; our proposed encoder, profiles and cost-aware single-model policy are not its original implementation.

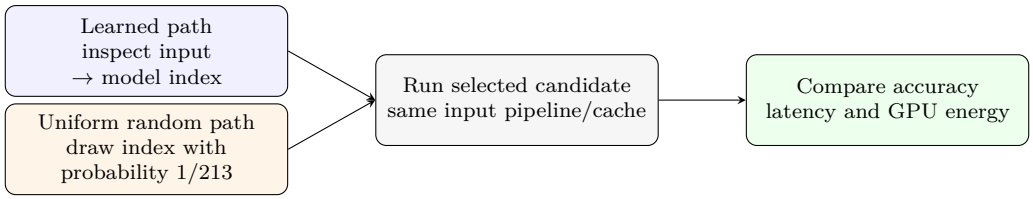
Deep Sets: optional extension, not required for a changing list. Our shared scorer evaluates each candidate independently. Deep Sets (Zaheer et al., NeurIPS 2017) could additionally summarize the available profiles without depending on their order, then feed that summary to the scorer. It supplies collection context, not automatic knowledge of a new model. The extra profile transformation, aggregation and scoring inputs can add computation, but latency depends on network size, caching and hardware; no overhead has been measured here. Independent scoring is the simpler first experiment because deleting B does not itself change A's correctness.

Required validation. Withhold entire candidate models during training, then introduce them through profiles. Test whether the frozen scorer remains accurate; otherwise fine-tune or retrain. Both designs require initial offline training and new-model profiling/calibration. Neither guarantees zero-retraining replacement, and neither has been implemented in this system.

4. Does the learned router beat uniform random selection?

Answer: yes on accuracy in all 24 settings at each batch size. At batch 64, learned accuracy is 81.86–85.64% versus 36.26% for uniform random; at batch 128 it is 82.25–85.39% versus 36.25%. Learned median latency is lower in all 24 settings at each batch; GPU energy is lower in 24/24 at batch 64 and 21/24 at batch 128. Uniform draws from all 213 candidates, including severe pruning/quantization failures. Beating it does not establish an advantage over a good static model.

Comparison with the previous table. Match objective, seed, batch and allowance to the 13 October correctness matrix. All 48 fresh learned accuracies equal the corresponding previous values, and fresh selection counts reproduce the frozen policies. Timings are remeasured, not copied. Each batch study comprises 495 full-test sweeps, with five trials per setting.



Reading the table. ML is learned routing; R is uniform random. Accuracy columns show ML / R percentages (five-trial means). In each cost cell the pair is latency saving % / GPU-energy saving %, not raw costs. δ is the allowed accuracy loss in percentage points (pp); $*$ marks learned mean accuracy failing its allowance against the 85.80% R101 FP32 reference. Every uniform row fails. L/E in Obj./seed denotes the latency/energy calibration objective.

Savings baseline. The Benchmark identity is the original 13 October validation-selected eligible static model, measured afresh per case. Saving is $100(C_{\text{benchmark}} - C_{\text{method}})/C_{\text{benchmark}}$ for repeat-median latency or energy. Positive means cheaper; negative means worse. Identity is the same for the paired batch-64/128 rows, but its measured cost differs by batch and case.

			Batch 64			Batch 128		
Obj./seed	δ	Benchmark	Acc. ML / R	ML: L / E	R: L / E	Acc. ML / R	ML: L / E	R: L / E
L/42	0.5	R101 FP16	84.93* / 36.36	+10.5 / +12.6	-505.8 / -161.6	85.23* / 36.15	+4.6 / +6.3	-215.4 / -87.6
L/42	1	R101 FP16	84.02* / 36.36	+19.5 / +22.3	-529.4 / -170.0	84.02* / 36.15	+23.0 / +26.7	-211.9 / -88.1
L/42	1.5	R101 FP16	84.02* / 36.36	+19.4 / +20.8	-555.5 / -168.4	84.02* / 36.15	+12.0 / +27.3	-286.2 / -83.5
L/42	2	R101 FP16	83.08* / 36.36	+45.0 / +45.9	-127.0 / -50.8	84.02 / 36.15	+32.2 / +33.0	-32.7 / -15.7
L/45	0.5	R101 FP16	85.52 / 36.67	+2.7 / +2.6	-151.4 / -81.2	84.93* / 37.28	+11.8 / +12.3	-28.3 / +10.6
L/45	1	R101 FP16	85.17 / 36.67	+6.7 / +6.3	-152.5 / -82.4	84.93 / 37.28	+12.0 / +12.2	-28.1 / +10.3
L/45	1.5	R101 FP16	84.58 / 36.67	+11.3 / +11.2	-152.2 / -82.8	83.93* / 37.28	+27.2 / +26.6	-28.2 / +10.1
L/45	2	R101 FP16	83.23* / 36.67	+34.8 / +34.5	-136.8 / -69.8	83.93 / 37.28	+11.1 / +11.6	-28.0 / +11.1
L/48	0.5	R101 FP16	85.41 / 35.76	+3.6 / +5.3	-142.8 / -93.4	85.19* / 35.34	+9.0 / +9.4	-17.1 / +9.6
L/48	1	R101 FP16	84.84 / 35.76	+12.6 / +14.9	-149.0 / -99.1	84.94 / 35.34	+16.2 / +16.7	-17.1 / +10.3
L/48	1.5	R50 FP16	84.84 / 35.76	-45.0 / -44.1	-312.7 / -233.1	84.94 / 35.34	-47.5 / -47.3	-106.0 / -58.9
L/48	2	R50 FP16	81.86* / 35.76	+43.4 / +43.2	-307.6 / -228.2	82.25* / 35.34	+24.7 / +27.5	-105.9 / -56.3
E/42	0.5	R101 FP16	85.23* / 36.36	+4.2 / +4.0	-126.7 / -51.2	85.39 / 36.15	+6.2 / +6.2	-32.8 / -15.6
E/42	1	R101 FP16	84.44* / 36.36	+24.0 / +24.1	-122.2 / -52.0	84.51* / 36.15	+25.1 / +25.0	-32.7 / -17.9
E/42	1.5	R101 FP16	84.44 / 36.36	+19.9 / +19.9	-386.6 / -125.4	84.51 / 36.15	+17.3 / +19.9	-212.2 / -86.1
E/42	2	R101 FP16	83.51* / 36.36	-35.4 / -8.8	-542.4 / -168.9	83.08* / 36.15	+33.0 / +36.9	-253.4 / -105.6
E/45	0.5	R101 FP16	85.27* / 36.67	+6.4 / +5.9	-149.2 / -80.2	84.51* / 37.28	+12.4 / +10.3	-16.3 / +18.6
E/45	1	R101 FP16	84.74* / 36.67	+9.4 / +9.3	-151.8 / -81.3	84.51* / 37.28	+19.2 / +19.3	-28.2 / +10.3
E/45	1.5	R101 FP16	84.34 / 36.67	+17.4 / +15.9	-146.0 / -80.0	83.93* / 37.28	+27.1 / +26.4	-28.2 / +10.0
E/45	2	R101 FP16	83.49* / 36.67	+29.0 / +29.0	-152.4 / -80.4	83.93 / 37.28	+27.1 / +26.8	-28.2 / +10.2
E/48	0.5	R101 FP16	85.64 / 35.76	+3.8 / +3.6	-139.7 / -93.2	85.39 / 35.34	+5.3 / +5.9	-17.1 / +10.0
E/48	1	R101 FP16	85.18 / 35.76	+8.3 / +7.7	-132.2 / -88.8	85.12 / 35.34	+14.3 / +14.5	-17.1 / +9.9
E/48	1.5	R50 FP16	84.66 / 35.76	-44.8 / -42.9	-303.8 / -224.5	85.12 / 35.34	-50.7 / -49.7	-105.9 / -58.1
E/48	2	R50 FP16	83.65* / 35.76	-15.7 / -13.1	-304.2 / -226.8	82.29* / 35.34	+18.7 / +21.3	-106.0 / -57.7

Shared random trials and uncertainty. Uniform controls are reused across objectives/allowances, not independent experiments per row. For seeds 42/45/48, accuracy mean ± sample standard deviation is respectively 36.36±2.64, 36.67±1.47, 35.76±2.12% at batch 64, and 36.15±2.07, 37.28±4.22, 35.34±2.57% at batch 128. Different row benchmarks explain why identical random measurements can have different savings.

Measurement boundary and caution. Both paths include fresh GPU input preparation, selected-model load/eviction, inference and synchronization with cache capacity two. Learned routing additionally executes its CNN; random schedule generation is setup, but dispatch lookup is timed. NVIDIA reported thermal slowdown/power capping and a fixed R101 FP32 sweep varied from about 15.4 to 58 seconds. These observations prevent attributing every timing difference to routing. All accuracy failures and actual measured outcomes are retained; the policies were not retuned using these test results.

5. Always using a fixed 80%-pruned model

Answer: lower cost, but unacceptable accuracy loss. These completed p80 controls reach 0.93–22.55% accuracy, versus 85.80% for unpruned R101 FP32. None meets a 0.5/1/1.5/2-pp allowance. Here p80 means the nominal 80% pruning setting, not a guarantee of 80% fewer operations; INT8 is 8-bit integer precision. Seven architectures, three precisions, four batches and five repeats plus references give 440 sweeps. ShuffleNet has no pruned variant.

Comparison with 13 October. Its correctness router reached 81.86–85.64% across batches 64/128, far above p80. Against the same *type* of fixed R101 FP32 reference, earlier routing latency/energy saving ranges were: b8 −336.2 to +61.1 / −160.7 to +60.4%; b32 +60.9 to +76.3 / +61.3 to +76.5%; b64 +61.5 to +87.4 / +63.6 to +87.5%; b128 +60.0 to +81.8 / +60.5 to +82.0%. These are separate ranges across policies, including accuracy failures, not one policy achieving every endpoint. P80 saves more cost in many rows, but sacrifices most classification accuracy.

Savings baseline. Table pairs are latency saving % / GPU-energy saving % versus fresh, same-batch R101 FP32, matching the earlier *vs-FP32* definition, not the routing table's eligible-static benchmark. Accuracy is the repeat mean; costs are repeat medians. Separate sessions and observed GPU throttling limit direct cross-study timing claims.

Architecture	Precision	Acc.%	b8: L% / E%	b32: L% / E%	b64: L% / E%	b128: L% / E%
EfficientNet-B0	FP32	22.49	+84.0 / +86.2	+91.5 / +93.0	+84.6 / +84.4	+84.6 / +84.3
EfficientNet-B0	FP16	22.55	+85.4 / +88.4	+93.4 / +94.8	+90.1 / +90.0	+90.3 / +90.3
EfficientNet-B0	INT8	2.78	+78.8 / +81.3	+91.0 / +93.7	+91.2 / +90.9	+91.3 / +91.2
MobileNetV3-Large	FP32	12.96	+89.3 / +91.7	+94.8 / +96.1	+92.9 / +92.8	+92.9 / +92.9
MobileNetV3-Large	FP16	12.98	+90.0 / +92.1	+95.6 / +97.0	+95.0 / +95.0	+95.1 / +95.0
MobileNetV3-Large	INT8	5.98	+86.7 / +88.7	+94.3 / +96.1	+95.4 / +95.4	+95.6 / +95.6
MobileNetV3-Small	FP32	20.94	+89.8 / +91.6	+95.6 / +97.0	+95.5 / +95.5	+95.6 / +95.5
MobileNetV3-Small	FP16	20.97	+90.3 / +92.3	+95.6 / +97.3	+96.3 / +96.4	+96.6 / +96.5
MobileNetV3-Small	INT8	10.09	+87.5 / +90.0	+94.2 / +96.4	+96.1 / +95.8	+96.4 / +96.5
ResNet-101	FP32	1.84	+79.5 / +83.1	+84.7 / +88.1	+86.1 / +86.3	+86.6 / +86.6
ResNet-101	FP16	1.84	+87.8 / +90.2	+93.5 / +95.0	+93.2 / +93.9	+92.7 / +92.7
ResNet-101	INT8	0.93	+85.6 / +88.7	+93.3 / +95.1	+94.3 / +94.6	+94.0 / +93.9
ResNet-18	FP32	5.07	+93.7 / +94.2	+94.5 / +95.6	+95.2 / +95.5	+95.7 / +95.7
ResNet-18	FP16	5.06	+95.6 / +96.3	+97.5 / +98.1	+96.7 / +96.6	+96.9 / +96.9
ResNet-18	INT8	4.97	+94.6 / +95.2	+96.6 / +97.4	+96.3 / +96.2	+96.5 / +96.6
ResNet-34	FP32	5.47	+91.5 / +92.4	+94.0 / +95.1	+95.1 / +95.4	+95.5 / +95.5
ResNet-34	FP16	5.45	+94.1 / +95.0	+96.4 / +97.2	+96.6 / +96.8	+96.8 / +96.9
ResNet-34	INT8	4.68	+92.9 / +94.4	+96.4 / +97.4	+96.6 / +96.8	+96.9 / +96.8
ResNet-50	FP32	6.01	+89.1 / +90.8	+92.0 / +92.8	+90.4 / +90.5	+90.8 / +90.7
ResNet-50	FP16	6.02	+92.7 / +94.0	+95.8 / +96.8	+94.4 / +94.4	+94.4 / +94.3
ResNet-50	INT8	4.70	+90.9 / +92.9	+95.3 / +96.5	+95.2 / +95.1	+95.2 / +95.2

Per-class strengths of the unpruned FP32 models

Class accuracy is recall on 100 test images per class. Values below use verified saved predictions, without new inference. All best/worst ties are shown; middle classes occupy ranks 50/51 (ties by class ID).

Architecture	Overall	Worst class	Best class(es)	Median-ranked classes
ResNet-18	79.16%	girl (45%)	motorcycle (96%)	kangaroo / lion (82%)
ResNet-34	81.82%	girl (48%)	motorcycle (100%)	rocket / telephone (84%)
ResNet-50	84.39%	girl (50%)	chimpanzee / skunk (98%)	hamster / leopard (86%)
ResNet-101	85.80%	girl (56%)	skunk (98%)	hamster / house (89%)
MobileNetV3-Small	62.38%	woman (30%)	road (93%)	fox / telephone (62%)
MobileNetV3-Large	81.34%	girl (50%)	bicycle / orange / road / tractor (96%)	house / leopard (83%)
EfficientNet-B0	82.73%	girl (53%)	road / wardrobe (97%)	table (83%); bee (84%)
ShuffleNetV2 x1.0	75.64%	otter (41%)	motorcycle (97%)	house / poppy (76%)

Complementarity example. Forest accuracy is 62% for R101, 71% for R18 and 76% for MobileNetV3-Large despite their lower overall scores. Their prior FP32 batch-32 times were 131.28,13.64,6.92 ms (static boundary, not full routing). This supports PERTINENCE's motivation, not proof of routing gains; the true class is unknown at inference.[2] The separate 71-row supplement covers all213 variants. Frozen numerical results/source hashes remain in `numerical_snapshot.json`.

[2]Shende et al., PERTINENCE, *IEEE Access*, 2026; paper.